Experiment No. 6
LOOPING STATEMENTS

I. OBJECTIVES

1. To employ different looping statements in C++ programs.
2. To differentiate and use looping statements in different applications.

II. LABORATORY EXERCISES

Encode the following program and compile them. Illustrate the output.
Note: if getch(); will not work on other C++ ide compiler replace it with _getch();
For www.onlinegdb.com editor and cxxdroid (mobile app) you may remove it.

Program no.1 (save as loopex1.cpp)

```
#include<iostream>
#include<windows.h>
#include<conio.h>
using namespace std;
int main ()
{
int num =1;
while (num<=5){
        cout<<"\n "<<num;
        Sleep(3000);
        num++;
}
_getch();
return 0;
}
```

Program No. 2 (save as loopex2.cpp)

```
#include<iostream>
#include<conio.h>
using namespace std;
int main ()
{
int x;
for (x =1; x<10;x++)
{       if (x ==5)
break;
        cout<< x<< "  ";
        }
cout << "\nBroke out of loop at x of " << x <<endl;
_getch();
return 0;
}
```

Program No 3. (Save as loopex3.cpp)

```
#include<iostream>
#include<conio.h>
using namespace std;
int main ()
{
int y, x=1, total =0;
while (x <= 10)
{           y=x * x;
            cout << y << endl;
            total +=y;
            ++x;  }
cout << "Total is " <<total <<endl;

_getch();
return 0;
}
```

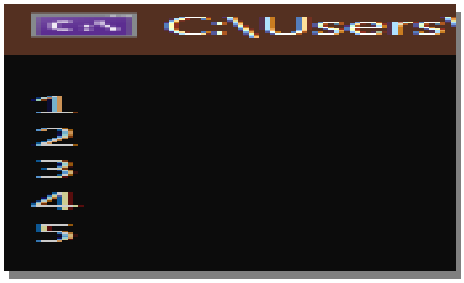
Program No 4. (save as loopex4.cpp)

```
#include<iostream>
#include<conio.h>
using namespace std;
int main ()
{
int x, y;
cout<<"\t\t\t\t Multiplication Table \n\n";
for(x=1;x<=10;x++)
{
            for(y=1;y<=10;y++)
            {
                        cout<<x*y<<"\t";
            }
            cout<<"\n";
}
_getch();
return 0;
}
```

III. Summary of Program Outputs

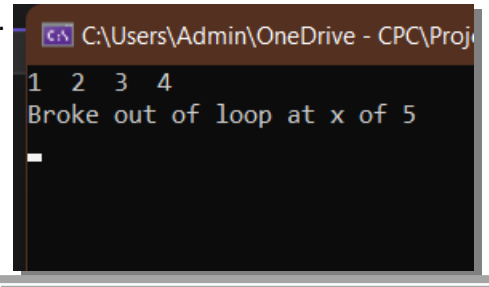
Direction: Demonstrate the corresponding output for each of the given programs. Give your observation and analysis for each of the problems.

1.



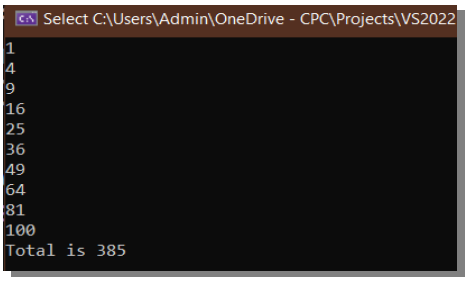
This Program is a c++ looping statement that used the while loop method to count from 1 to 5 with an interval of 3 seconds. It used “sleep” to go into a condition of inactivity for a specified period of time.

2.



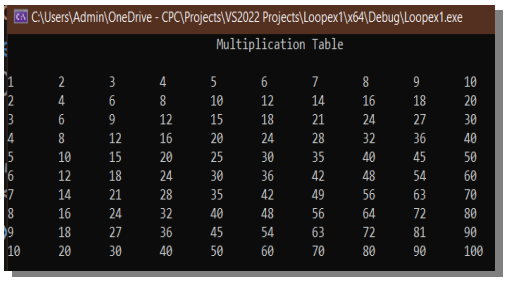
This Program is a c++ looping statement that used the for-loop method to produce a sequence of numbers and to test the value of x if it’s reached 5, if the x value reached 5 “Broke out of the loop at x of 5” will be displayed. Or if a variable's value is set to 1, the program will show that it will grow by 1 up to a maximum of 10, and if the value is set to 5, the loop will be broken.

3.



This Program is a c++ looping statement that used while loop method to display the sum or the total value of the incremeneted x and y.

4.



This Program is a c++ looping statement that used the double for loop method (Loop inside a loop) to display the multiplication table vertically from 1 up to 10. This program is just like the program n0. 3 but has a complicated method it uses a double for loop instead of the while loop which is much simpler.

IV. Supplementary Problems

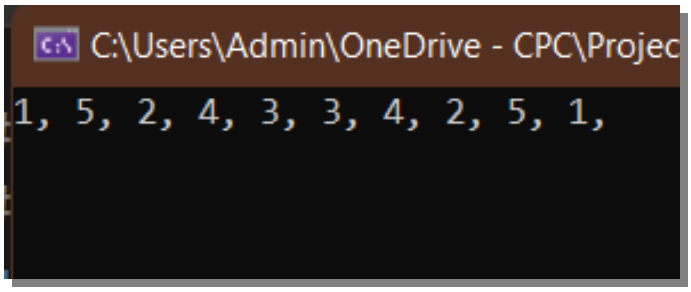
1. Write a program which produces the given sequence numbers (in alternative arrangement).

   1, 5, 2, 4, 3, 3, 4, 2, 5, 1,

```
#include <iostream>
#include <conio.h>
using namespace std;
int main()
{

   int a = 1;
   int b = 5;
   for (a, b; a <= 5; a++, b--)
   {
      cout << a << ", " << b << ", ";
   }
   _getch();
   return 0;
}
```

OUTPUT:



2. Write a program that reverses the input number n. Formulate an equation to come up with the answer. The program will continue to ask a number to reverse till the user enter '0' or to quit.

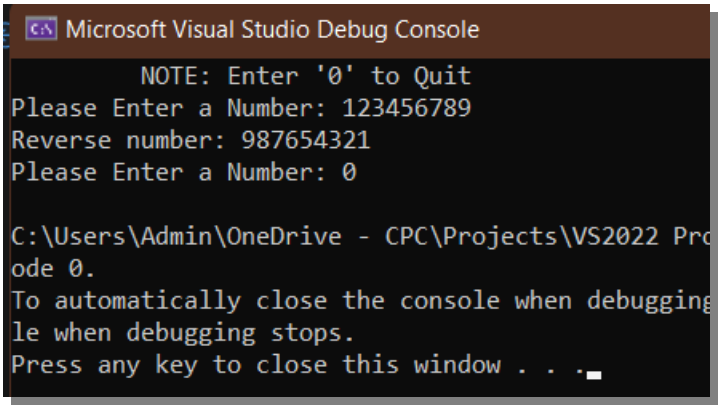
   Sample output:

   Enter a number: 1238
   Reverse number: 8321

```
#include<iostream>
#include <conio.h>
using namespace std;
int main()
{
   int num, remain, reverse = 0;
   cout << "\t NOTE: Enter '0' to Quit\n";
start:
   cout << "Please Enter a Number: ";
   cin >> num;
   if (num == 0)
```

```
        goto stop;
    while (num != 0)
    {
        remain = num % 10;
        reverse = remain + (reverse * 10);
        num = num / 10;
    }
    cout << "Reverse number: " << reverse;
    cout << endl;
    goto start;
    _getch();
stop:
    return 0;
}
```

OUTPUT:



3. Write a program to input a number then output the Fibonacci series up to the inputted number series.

Enter a number: 10
Fibonacci series: 01 1 2 3 5 8 13 21 34

```
#include<iostream>
#include <conio.h>
using namespace std;
int main()
{
    int num, s1 = 0, s2 = 1, nt = 0;

    cout << "Enter a number: ";
    cin >> num;
    cout << "Fibonacci Series: ";

    for (int a = 1; a <= num; ++a) {
    if (a == 1) {
    cout << s1 << ", ";}
    if (a == 2) {
    cout << s2 << ", ";}
```

```
    nt = s1 + s2;
    s1 = s2;
    s2 = nt;
cout << nt << ", ";
    }
    _getch();
    return 0;
}
```

OUTPUT:

```
C:\Users\Admin\OneDrive - CPC\Projects\VS2022 Projects\practiceprob\Debug\practiceprob.exe
Enter a number: 15
Fibonacci Series: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987,
```

V. Conclusion

I learned that in C++, a looping statement is used to repeatedly run a certain piece of code until a specified condition is met. You can employ a variety of looping statement techniques. The first is the for loop, which is used in C++ programming to repeatedly run a certain execution a certain number of times. This makes it easier for us to write code since it combines everything into a single line. When you want a statement to run constantly until the provided condition is true, you may use the second option, the while loop. With a for loop, you can predict how many times the loop will run, but a while loop is unpredictable. When using a while loop, you won't know how many times the loop will be executed until the program ends when the condition is not met. The last one is the Do/While loop, which differs from the for and while loops primarily in that the condition is tested at the end of the loop. In a do-while loop, the condition is evaluated after the loop's body has been completed. As a result, even if the condition is not met, the statement will always be performed at least once.